Software Engineering LAB 4

NAME: Ishanika Vinuthan
SRN: PES2UG24CS911
CLASS: 5B

UPDATED CODE

```
import random
import pygame
from game.text_box import TextBox



class GameEngine:
   def __init__(self, width, height):
       self.width = width
       self.height = height
       self.secret_number = random.randint(1, 100)

       # Task 4: Maximum attempts
       self.max_attempts = 7

       self.attempts = 0

       # Task 2: Dynamic range
       self.low_bound = 1
       self.high_bound = 100

       # Task 3: Guess history
       self.guess_history = []

       self.feedback_msg = "Enter a number between 1 and 100"
       self.feedback_color = (220, 220, 220)

       self.game_won = False

       # Task 4: Game over state
```

```
        self.game_over = False

        self.input_box = TextBox(width // 2 - 110, 150, 120, 48)
        self.submit_btn = pygame.Rect(width // 2 + 25, 150, 100, 48)

        self.font_title = pygame.font.SysFont(None, 42)
        self.font_medium = pygame.font.SysFont(None, 28)
        self.font_btn = pygame.font.SysFont(None, 26)
        self.font_history = pygame.font.SysFont(None, 22)

    def submit_guess(self):
        # Do not accept guesses after winning or game over
        if self.game_won or self.game_over:
            return

        # Task 1: Validate empty input
        if not self.input_box.text.strip():
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 200, 80)
            return

        guess = int(self.input_box.text)

        # Only valid guesses increase attempts
        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            # Task 2: Update lower boundary
            self.low_bound = guess

            self.feedback_msg = f"TOO LOW! (Guess was {guess})"
            self.feedback_color = (80, 160, 240)

            # Task 3: Add to history
            self.guess_history.append(
                (guess, "TOO LOW", (80, 160, 240))
            )
```

```
    elif guess > self.secret_number:
        # Task 2: Update upper boundary
        self.high_bound = guess

        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)

        # Task 3: Add to history
        self.guess_history.append(
            (guess, "TOO HIGH", (240, 100, 80))
        )

    else:
        self.feedback_msg = (
            f"CORRECT! Found in {self.attempts} attempts."
        )
        self.feedback_color = (80, 220, 90)
        self.game_won = True

        # Task 3: Record correct guess
        self.guess_history.append(
            (guess, "CORRECT", (80, 220, 90))
        )

    # Keep only the latest 5 guesses
    if len(self.guess_history) > 5:
        self.guess_history.pop(0)

    # Task 4: Trigger GAME OVER after 7 incorrect guesses
    if not self.game_won and self.attempts >= self.max_attempts:
        self.game_over = True

        self.feedback_msg = (
            f"GAME OVER! The secret number was {self.secret_number}."
        )
        self.feedback_color = (240, 100, 100)
```

```
    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0

        # Task 2: Reset range
        self.low_bound = 1
        self.high_bound = 100

        # Task 3: Reset history
        self.guess_history.clear()

        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)

        self.game_won = False

        # Task 4: Reset game over
        self.game_over = False

        self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()

            elif event.key == pygame.K_r and (
                self.game_won or self.game_over
            ):
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
```

```
        pass

    def render(self, screen):
        screen.fill((30, 34, 42))

        # Title
        title_surf = self.font_title.render(
            "Number Guessing Arena",
            True,
            (245, 245, 245)
        )
        screen.blit(
            title_surf,
            (
                self.width // 2 - title_surf.get_width() // 2,
                35
            )
        )

        # Attempts
        attempts_surf = self.font_medium.render(
            f"Attempts: {self.attempts}/{self.max_attempts}",
            True,
            (180, 185, 195)
        )
        screen.blit(
            attempts_surf,
            (
                self.width // 2 - attempts_surf.get_width() // 2,
                95
            )
        )

        # Dynamic range
        range_surf = self.font_medium.render(
            f"Current Possible Range: "
            f"{self.low_bound} - {self.high_bound}",
            True,
```

```
            (200, 200, 200)
        )
        screen.blit(
            range_surf,
            (
                self.width // 2 - range_surf.get_width() // 2,
                120
            )
        )

        # Input box
        self.input_box.render(screen)

        # Submit button
        pygame.draw.rect(
            screen,
            (50, 150, 80),
            self.submit_btn,
            border_radius=6
        )

        pygame.draw.rect(
            screen,
            (220, 220, 220),
            self.submit_btn,
            width=2,
            border_radius=6
        )

        btn_text = self.font_btn.render(
            "SUBMIT",
            True,
            (255, 255, 255)
        )

        screen.blit(
            btn_text,
            (
```

```
            self.submit_btn.centerx - btn_text.get_width() // 2,
            self.submit_btn.centery - btn_text.get_height() // 2
        ),
    )

    # ------------------------------------------------
    # Task 4: Restart prompt ABOVE feedback message
    # ------------------------------------------------
    if self.game_won or self.game_over:
        restart_surf = self.font_medium.render(
            "Press [R] to Start a New Game",
            True,
            (255, 220, 80)
        )

        screen.blit(
            restart_surf,
            (
                self.width // 2 - restart_surf.get_width() // 2,
                205
            )
        )

    # Feedback message
    feedback_surf = self.font_medium.render(
        self.feedback_msg,
        True,
        self.feedback_color
    )

    screen.blit(
        feedback_surf,
        (
            self.width // 2 - feedback_surf.get_width() // 2,
            235
        )
    )
```

```
        # ------------------------------------------------
        # Task 3: Smaller Guess History panel
        # ------------------------------------------------
        history_x = self.width // 2 - 135
        history_y = 270
        history_width = 270
        history_height = 135

        pygame.draw.rect(
            screen,
            (42, 47, 57),
            (
                history_x,
                history_y,
                history_width,
                history_height
            ),
            border_radius=8
        )

        pygame.draw.rect(
            screen,
            (100, 105, 115),
            (
                history_x,
                history_y,
                history_width,
                history_height
            ),
            width=2,
            border_radius=8
        )

        # History title
        history_title = self.font_history.render(
            "Guess History (Last 5)",
            True,
            (235, 235, 235)
```

```
        )

        screen.blit(
            history_title,
            (
                self.width // 2 - history_title.get_width() // 2,
                history_y + 7
            )
        )

        # History entries
        for index, (guess, result, color) in enumerate(
            self.guess_history
        ):

            if result == "TOO LOW":
                symbol = "v"
            elif result == "TOO HIGH":
                symbol = "^"
            else:
                symbol = "*"

            history_text = self.font_history.render(
                f"{guess}  {symbol}  {result}",
                True,
                color
            )

            screen.blit(
                history_text,
                (
                    history_x + 25,
                    history_y + 34 + index * 19
                )
            )

        # GAME OVER extra information
        if self.game_over:
```

```
            reveal_surf = self.font_history.render(
                f"Secret number: {self.secret_number}",
                True,
                (240, 220, 220)
            )

            screen.blit(
                reveal_surf,
                (
                    self.width // 2 - reveal_surf.get_width() // 2,
                    history_y + history_height + 8
                )
            )
```